ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)

Assignment

CSE 4307: Database Management Systems

CLI tool of B-tree

Make a CLI tool in C, that simulates step-by-step value insertion in a B-Tree.

Submitted By:

Name: **Mushfeka Ferdousi**

StudentID: **230042126**

Department: **CSE**

Program: **SWE**

Submitted To:

Md. Tariquzzaman

Source Code:

Users > zen > sem03 > dbms lab > C test.c > printPaths(BTreeNode *, char *)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

int M;

typedef struct BTreeNode {
    int *keys;
    struct BTreeNode **children;
    int num_keys;
    bool is_leaf;
} BTreeNode;

BTreeNode* createNode(bool is_leaf) {
    BTreeNode* node = (BTreeNode*)malloc(sizeof(BTreeNode));
    node->keys = (int*)malloc(sizeof(int) * M);
    node->children = (BTreeNode**)malloc(sizeof(BTreeNode*) * (M + 1));
    node->num_keys = 0;
    node->is_leaf = is_leaf;
    for (int i = 0; i <= M; i++) node->children[i] = NULL;
    return node;
}

void printPaths(BTreeNode* node, char* path_prefix) {
    if (!node) return;
    char current_node_str[200] = "[";
    for (int i = 0; i < node->num_keys; i++) {
        char temp[20];
        sprintf(temp, "%d%s", node->keys[i], (i == node->num_keys - 1) ? "" : "|");
        strcat(current_node_str, temp);
    }
    strcat(current_node_str, "]");
    char new_path[1000];
    if (strlen(path_prefix) == 0) strcpy(new_path, current_node_str);
    else sprintf(new_path, "%s -> %s", path_prefix, current_node_str);
    if (node->is_leaf) {
        printf("%s\n", new_path);
    } else {
        for (int i = 0; i <= node->num_keys; i++) {
            printPaths(node->children[i], new_path);
        }
```

```
    }
}

BTreeNode* split(BTreeNode* node, int* promo_key) {
    int mid = (M - 1) / 2;
    *promo_key = node->keys[mid];
    BTreeNode* newNode = createNode(node->is_leaf);
    newNode->num_keys = node->num_keys - mid - 1;
    for (int i = 0; i < newNode->num_keys; i++) {
        newNode->keys[i] = node->keys[i + mid + 1];
    }
    if (!node->is_leaf) {
        for (int i = 0; i <= newNode->num_keys; i++) {
            newNode->children[i] = node->children[i + mid + 1];
        }
    }
    node->num_keys = mid;
    return newNode;
}

BTreeNode* insertRecursive(BTreeNode* node, int key, int* promo_key, BTreeNode** promo_child) {
    int i = 0;
    while (i < node->num_keys && key > node->keys[i]) i++;

    if (node->is_leaf) {
        for (int j = node->num_keys; j > i; j--) node->keys[j] = node->keys[j - 1];
        node->keys[i] = key;
        node->num_keys++;
    } else {
        BTreeNode* split_node = insertRecursive(node->children[i], key, promo_key, promo_child);
        if (split_node) {
            int k = *promo_key;
            BTreeNode* c = split_node;
            for (int j = node->num_keys; j > i; j--) {
                node->keys[j] = node->keys[j - 1];
                node->children[j + 1] = node->children[j];
            }
            node->keys[i] = k;
            node->children[i + 1] = c;
            node->num_keys++;
```

```c
            node->num_keys++;
        }
    }

    if (node->num_keys >= M) {
        return split(node, promo_key);
    }
    return NULL;
}

BTreeNode* insert(BTreeNode* root, int key) {
    if (!root) {
        root = createNode(true);
        root->keys[0] = key;
        root->num_keys = 1;
        return root;
    }
    int promo_key;
    BTreeNode* promo_child;
    BTreeNode* split_node = insertRecursive(root, key, &promo_key, &promo_child);
    if (split_node) {
        BTreeNode* newRoot = createNode(false);
        newRoot->keys[0] = promo_key;
        newRoot->children[0] = root;
        newRoot->children[1] = split_node;
        newRoot->num_keys = 1;
        return newRoot;
    }
    return root;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int values[n];
    for (int i = 0; i < n; i++) scanf("%d", &values[i]);
    scanf("%d", &M);
    BTreeNode* root = NULL;
    for (int i = 0; i < n; i++) {
        root = insert(root, values[i]);
```

```c
        printf("Step %d: Insert %d\n", i + 1, values[i]);
        printPaths(root, "");
        printf("\n");
    }
    return 0;
}
```

Output1:

```
10
1 2 3 4 5 6 7 8 9 10
3
Step 1: Insert 1
[1]

Step 2: Insert 2
[1|2]

Step 3: Insert 3
[2] -> [1]
[2] -> [3]

Step 4: Insert 4
[2] -> [1]
[2] -> [3|4]

Step 5: Insert 5
[2|4] -> [1]
[2|4] -> [3]
[2|4] -> [5]

Step 6: Insert 6
[2|4] -> [1]
[2|4] -> [3]
[2|4] -> [5|6]

Step 7: Insert 7
[4] -> [2] -> [1]
[4] -> [2] -> [3]
[4] -> [6] -> [5]
[4] -> [6] -> [7]

Step 8: Insert 8
[4] -> [2] -> [1]
[4] -> [2] -> [3]
[4] -> [6] -> [5]
[4] -> [6] -> [7|8]

Step 9: Insert 9
[4] -> [2] -> [1]
[4] -> [2] -> [3]
[4] -> [6|8] -> [5]
[4] -> [6|8] -> [7]
[4] -> [6|8] -> [9]
```

```
Step 10: Insert 10
[4] -> [2] -> [1]
[4] -> [2] -> [3]
[4] -> [6|8] -> [5]
[4] -> [6|8] -> [7]
[4] -> [6|8] -> [9|10]
```

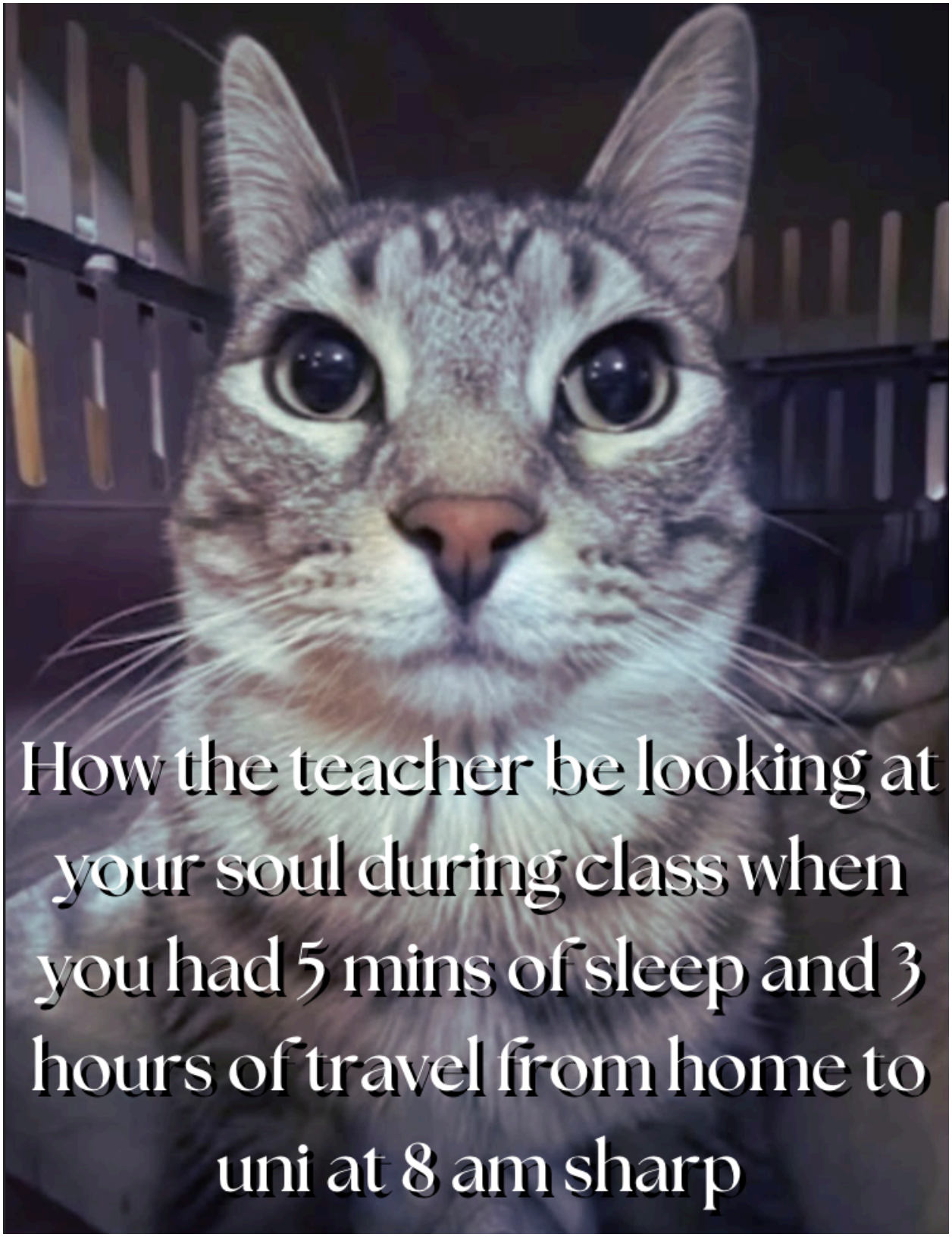
Output2:

```
11
7 4 10 3 2 9 12 18 16 21 22
4
Step 1: Insert 7
[7]

Step 2: Insert 4
[4|7]

Step 3: Insert 10
[4|7|10]

Step 4: Insert 3
[4] -> [3]
[4] -> [7|10]

Step 5: Insert 2
[4] -> [2|3]
[4] -> [7|10]

Step 6: Insert 9
[4] -> [2|3]
[4] -> [7|9|10]

Step 7: Insert 12
[4|9] -> [2|3]
[4|9] -> [7]
[4|9] -> [10|12]

Step 8: Insert 18
[4|9] -> [2|3]
[4|9] -> [7]
[4|9] -> [10|12|18]

Step 9: Insert 16
[4|9|12] -> [2|3]
[4|9|12] -> [7]
[4|9|12] -> [10]
[4|9|12] -> [16|18]

Step 10: Insert 21
[4|9|12] -> [2|3]
[4|9|12] -> [7]
[4|9|12] -> [10]
[4|9|12] -> [16|18|21]
```

```
Step 11: Insert 22
[9] -> [4] -> [2|3]
[9] -> [4] -> [7]
[9] -> [12|18] -> [10]
[9] -> [12|18] -> [16]
[9] -> [12|18] -> [21|22]
```

Meme:

in closure calculation
bruteforce approach
basket algorithm
absolutely useless


How the teacher be looking at your soul during class when you had 5 mins of sleep and 3 hours of travel from home to uni at 8 am sharp